REPORT FINAL TASK 05 MATTIA MUNGO

In this task the goal was to complete and validate the entire pipeline compilation, from the C++ source to the execution of the assembly x86-64 code. The first thing I did was to complete the empty // TDB sections in the codegen.cpp file, as requested. I mapped the arithmetic operators, like add, sub, imul ecc. The relational operators are translated in comparisons followed by conditional jumps, like jl, jle, jg, jge, ecc.

Then, I implemented a support function for the resolution of the operands, which distinguishes correctly the immediate constants, variables and temporaries. The variables are allocated in the .ss section of the assembly file; when the program uses the print of integers, also the support buffers are automatically generated. The structure of the resultant output.asm file includes the sections .bss, .data and .text, with a starting point _start. The creation of the assignment instructions was done by loading the operands in the general registers and by memorizing the results in the associated location of memory of the destination variable. The comparison instructions produce a cmp operation, followed by a conditional jump to the appropriate label. This allows them to correctly represent the sequential constructs, conditionals and cycles in the final assembly code. Furthermore, the printing of integers was realized by an assembly routine dedicated, which converts the value contained in rax in an ASCII representation and it sends it to a standard output with the system call write. This routine is included in the assembly file only if necessary; the same happens with the printings of the strings. The program always ends with the system call exit, which guarantees a correct conclusion of the execution.

Talking personally, a significant part of the work was about the execution of the assembly code generated in a macOS environment. Since macOS doesn't support the execution of binaries ELF x86-64 with NASM and ld, for me it was necessary to use Docker in order to create a Linux environment compatible. The output.asm file was generated by the compiler and was set up inside a container Ubuntu 22.04 and assembled using NASM. The command nasm produces an output.o file, an object file in ELF format, which contains binary data unreadable as a text. After that, the object file was connected to the ld to produce an executable Linux x86-64 called program. The execution of this binary inside of the Docker container produces the correct output.

With the standard read.txt file:

```
int x;
x = 5;

int y;
y = 10;

prints ("Hello world!");
print(x);
print(y);
```

the execution of the program generates the correct result.

The output can be seen directly in the terminal:

```
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
Hello world!
5
10
```

or it can be redirected in a text file called result.txt, in order to verify its content:

```
Hello world!
5
10
```

Doing this, the entire chain of compilation was validated, from the source pseudo-C++ to the execution of the generated assembly code, even in my macOS environment, thanks to the use of Docker.

We can also see how it works well with another example. If we change the read.txt file to this:

```
a = 10 + 5;
b = a * 2;
c = b - 10;

print(a);
print(b);
print(c);

if (c > 10) {
    print(c);
} else {
    print(0);
}

i = 0;
while (i < 3) {
    print(i);
    i = i + 1;
}
```

We can see that the output.asm is:

```
section .bss
        digitSpace resb 100
        digitSpacePos resb 8

        T4 resb 8
        T2 resb 8
        T3 resb 8
        T1 resb 8
section .data
section .text
        global _start

_start:
section .bss
        Vi resb 8
        Vc resb 8
        Vb resb 8
        Va resb 8
section .text
        mov rax, 10
        mov rbx, 5
        add rax, rbx
        mov [T1], rax
        mov rax, [T1]
        mov [Va], rax
        mov rax, [Va]
        mov rbx, 2
        imul rax, rbx
        mov [T2], rax
        mov rax, [T2]
        mov [Vb], rax
        mov rax, [Vb]
        mov rbx, 10
        sub rax, rbx
        mov [T3], rax
        mov rax, [T3]
        mov [Vc], rax
        mov rax, [Va]
        call _print_num
        mov rax, [Vb]
        call _print_num
        mov rax, [Vc]
        call _print_num
        mov rax, [Vc]
        mov rbx, 10
        cmp rax, rbx
        jg L1
        jmp L3
L1:
        mov rax, [Vc]
        call _print_num
        jmp L2
L3:
        mov rax, 0
        call _print_num
L2:
        mov rax, 0
        mov [Vi], rax
L4:
        mov rax, [Vi]
        mov rbx, 3
        cmp rax, rbx
        jl L5
        jmp L6
```

```
L5:
        mov rax, [Vi]
        call _print_num
        mov rax, [Vi]
        mov rbx, 1
        add rax, rbx
        mov [T4], rax
        mov rax, [T4]
        mov [Vi], rax
        jmp L4
L6:
        mov rax, 60
        mov rdi, 0
        syscall


_print_num:
        push rbx
        push rcx
        push rdx
        push rsi
        push rdi
        push r8
        cmp rax, 0
        jne ._pn_not_zero
        mov byte [digitSpace+99], '0'
        mov rax, 1
        mov rdi, 1
        lea rsi, [digitSpace+99]
        mov rdx, 1
        syscall
        jmp ._pn_newline
._pn_not_zero:
        mov rbx, 0
        cmp rax, 0
        jge ._pn_positive
        neg rax
        mov rbx, 1
._pn_positive:
        lea rcx, [digitSpace+100]
        mov r8, 10
._pn_loop:
        xor rdx, rdx
        div r8
        add dl, '0'
        dec rcx
        mov [rcx], dl
        test rax, rax
        jnz ._pn_loop
        cmp rbx, 1
        jne ._pn_write
        dec rcx
        mov byte [rcx], '-'
._pn_write:
        mov rax, 1
        mov rdi, 1
        mov rsi, rcx
        lea rdx, [digitSpace+100]
        sub rdx, rcx
        syscall
._pn_newline:
        sub rsp, 8
        mov byte [rsp], 10
        mov rax, 1
        mov rdi, 1
        mov rsi, rsp
        mov rdx, 1
        syscall
        add rsp, 8
        pop r8
        pop rdi
        pop rsi
        pop rdx
        pop rcx
        pop rbx
        ret
```

And that the result.txt is:



As expected.

OPTIMIZATION OF THE CODE:

In order to fully complete the task and the project, I also did a phase of optimization of the intermediate code, coherent with the techniques presented in the slides; it was realized fully at level IR, without changing the parser, AST or code generation. All of the changes are concentrated in the ir.cpp file, in order to preserve the original structure of the pipeline of compilation and to respect the project requirements.

The first thing I did was the constant folding; during the generation of intermediate code, inside of the function exec_expr, a control was added on the binary nodes in order to verify that both of the operands are numerical constants. In this case, the expression is evaluated directly at compile-time and the calculator value is given as a constant, in order to avoid the creation of temporary variables and unuseful IR instructions. This optimization reduces immediately the IR dimensions and simplifies the produced code in the following phases.

After that, I introduced an optimization of post-generation of the IR, executed before the restitution of the GeneratedIR structure. In this phase, it's applied the constant propagation is applied, keeping a map of the variables of which the values are given at compile-time. When a variable with constant value is used as an operand in a successive instruction, its value is substituted directly with the constant, making possible other simplifications and new cases of constant folding.

In the same phase, a form of dead code elimination was implemented to the constant conditions. The comparisons instructions are analyzed and, if both of the operands are constant, the condition is evaluated at compile-time. If the condition is always true, the comparison instruction is substituted with an unconditional jump to the destination label. This allows to remove useless comparison and to simplify the control flow when the program's behaviour is statistically determinable.

Furthermore, a common subexpression elimination at basic block level was implemented. During the analysis of IR, the arithmetic expressions are normalized and memorized in a

map that connects the canonic expression to the temporary value that has the result. If the same expression appears again in the same basic block, the previous results is reused and the original instruction is transformed in a simple assignment. The map of expressions is reset in correspondence of the labels and jumps, in order to guarantee that the optimization is only inside of the basic single block, as the theory said.

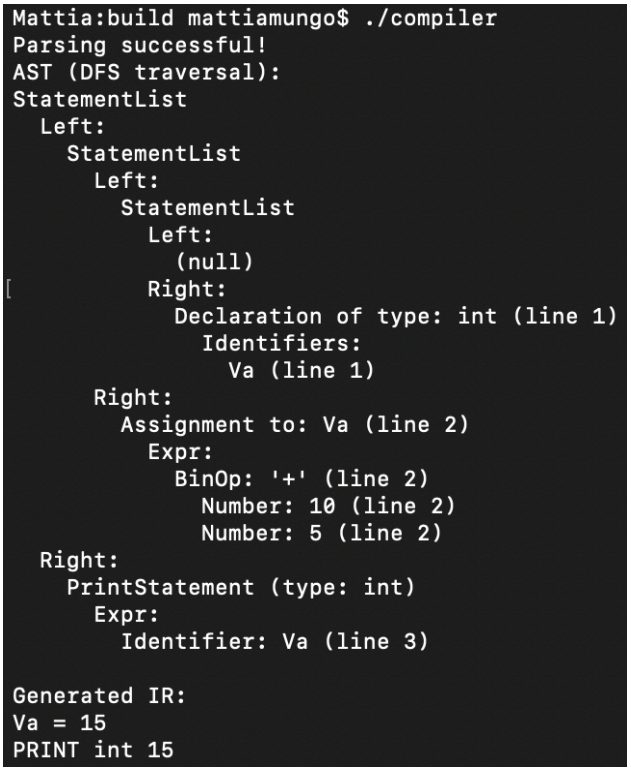
At the end, also the printing instructions are involved in the constant propagation, substituting the variables with constant values when possible, in order to simplify the IR even more before the generation of the assembly code.

All of these operations make the IR simpler, which leads to a shorter and more linear assembly code.

A possible example is this one:

the assignment of an expression like a = 10 + 5; in the not-optimized version of the compiler, this expression generates an IR instruction with a temporary and the operation of the sum is executed at runtime. With the optimized version, the expression is evaluated directly during the generation of the intermediate code, producing a simplified instruction a = 15. Doing like this, we can delete useless temporary variables and reduce the number of IR instructions.

The screenshot shows the operation of a sum in an optimized version, where the result is already solved.



```
Mattia:build mattiamungo$ ./compiler
Parsing successful!
AST (DFS traversal):
StatementList
  Left:
    StatementList
      Left:
        StatementList
          Left:
            (null)
          Right:
            Declaration of type: int (line 1)
              Identifiers:
                Va (line 1)
      Right:
        Assignment to: Va (line 2)
          Expr:
            BinOp: '+' (line 2)
              Number: 10 (line 2)
              Number: 5 (line 2)
  Right:
    PrintStatement (type: int)
      Expr:
        Identifier: Va (line 3)

Generated IR:
Va = 15
PRINT int 15
```